

Validate Before You Apply: An Action-Validation Layer for LLM Tutors that Act on Learner Artifacts in Computer Networks Education

Anonymous Author(s)

Submission to IEEE TALE 2026 — omitted for double-blind review

Abstract—Large language models (LLMs) promise adaptive tutoring, but when a tutoring agent can also act on the learner’s working artifact, hallucination acquires a new cost: an incorrect action silently corrupts the very object the student is reasoning about. We present CAPA8, a browser-based environment for computer networks education that couples a conversational LLM tutor with an interactive network-topology simulator the tutor can read and modify. Its central design contribution is a pedagogical reliability layer: every model-proposed modification is expressed in a closed action grammar, parsed, and validated against the live topology state before it is applied, with ambiguous requests resolved by rule-based clarification dialogues that bypass the LLM entirely. Adaptive scaffolding is provided by a Level $\times$ Focus mode matrix that reconfigures the tutor from step-by-step guidance to a technical peer. We make no claim about learning outcomes; the contribution is the design and its technical reliability. Because referential validation rejects any dangling reference by construction, our benchmark of 900 model interactions across three deployment scenarios (two local open-weight models and a cloud-hosted model) does not test *whether* invalid actions are caught but measures what makes the layer necessary: how often, and in which classes, models of differing capability emit invalid or spurious actions. Even a 70B cloud model proposes invalid actions on every run, while smaller models also emit spurious-but-well-formed ones that structural validation cannot see and a human-in-the-loop preview must cover. A classroom study is planned as future work.

Index Terms—large language models, intelligent tutoring systems, scaffolding, computer networks education, hallucination mitigation, human–AI collaboration

I. INTRODUCTION

Computer networking is taught through abstractions—addressing, subnetting, routing, gateways—whose consequences only become visible when a concrete topology misbehaves. Professional simulators such as Cisco Packet Tracer [14] and emulation environments such as Mininet [15] give students that concreteness, but they carry steep learning curves, and none of them explains *why* the ping the student just issued never arrived. The explanatory gap is usually filled by an instructor, who does not scale, or lately by a general-purpose chatbot, which cannot see the student’s diagram and therefore reasons about a network that does not exist.

LLMs have shown clear potential as adaptive explainers in engineering education [1], [2]. The obvious next step—letting the tutor not only explain the student’s artifact but also act on it—introduces a reliability problem that is qualitatively different from textual hallucination [4]. A fabricated sentence

can be challenged; a fabricated *action* (connecting a PC to a router that does not exist, duplicating an IP address, “fixing” a missing gateway by adding a spurious cable) rewrites the learning artifact itself. Novices are the least equipped to detect such corruption: they cannot tell whether the inconsistency they now observe is their own misconception or the tool’s error, which undermines both trust and the diagnostic reasoning the exercise was meant to train.

CAPA8 is a no-install, browser-based environment that addresses this problem by treating the LLM as an *untrusted proposer*. The model never mutates the topology directly. It emits proposals in a closed action grammar; a deterministic pipeline classifies the student’s intent before any model call, constrains the prompt, parses the response, validates every proposed action against the live graph state, and routes batches through a human-confirmable preview with single-step undo. The same pipeline carries the pedagogy: a Level $\times$ Focus mode matrix adapts the depth and register of explanations, and a topology analyzer injects the same misconfiguration warnings the student sees on the canvas into the model’s context, so tutor and student literally look at the same network.

This paper makes three contributions: (1) the design of a validate-before-apply action pipeline as a pedagogical reliability mechanism for LLM tutors that operate on learner artifacts; (2) an adaptive scaffolding scheme that operationalizes tutoring styles as a selectable Level $\times$ Focus matrix combined with intent-dependent decoding temperature; and (3) a preliminary technical evaluation—unit-test coverage of the deterministic pipeline plus a reliability benchmark over three deployment scenarios with an ablation of the prompt-side mechanisms—that, rather than re-deriving the tautology that a referential-integrity check rejects dangling references, quantifies the premise that makes such a check worthwhile: how often, and in which classes, LLMs of varying capability emit invalid or spurious actions. We evaluate reliability, not learning; comprehension and scaffolding effects await the classroom study (§VI).

II. RELATED WORK

LLMs in computing education: Kasneci et al. [1] map the opportunities and risks of LLMs across education, identifying hallucination and overreliance as core concerns. Within computing education, Denny et al. [2] survey the post-GPT landscape, and CodeHelp [3] demonstrates guardrails that keep

an LLM helper from emitting full solutions in programming classes. CodeHelp’s guardrails shape the *text* a model returns; CAPA8 extends the guardrail idea to *state-changing actions*, where the failure mode is not an unwanted answer but a corrupted artifact.

Scaffolding and intelligent tutoring systems: The tutor configurations in CAPA8 instantiate classic scaffolding theory: graduated assistance within the learner’s zone of proximal development [5], [6], software realizations of structuring and problematizing supports [7], and the ITS tradition of step-based adaptive dialogue [8]. CAPA8 differs from end-to-end LLM tutors in deliberately keeping parts of the dialogue deterministic: ambiguity resolution and misconfiguration detection are rule-based, reserving the LLM for explanation and proposal generation.

Constraining LLMs that act: Hallucination surveys [4] motivate a growing toolbox of output constraints, from programmable conversational rails to constrained semantic decoding [9]. Letting models *act* through tools is by now standard—reason-and-act loops [11] and large function-calling repertoires [12]—and such agents are increasingly placed in educational roles [13]. These mechanisms, including native JSON and function-calling modes, constrain the *syntax* of a call at generation time; they are stateless with respect to the application and cannot know that *Router5* is absent from *this* graph or that an address is already in use. CAPA8 adds a complementary, state-dependent stage: semantic validation of each proposed action against the live topology (referential integrity, duplicate addresses, type normalization), plus a human confirmation step consistent with human–AI interaction guidelines [10]. Validate-before-apply is thus not novel in the abstract; our contribution is its instantiation as a pedagogical reliability layer in an artifact-centric tutor, where a rejected action prevents a corrupted *learning* object rather than a failed API call. To our knowledge this has not been reported for network-education tools, whose mainstream representatives [14], [15] offer no integrated conversational tutoring at all.

III. SYSTEM DESIGN

A. Architecture Overview

CAPA8 is a dependency-light web app (vanilla ES6 modules, no build step) with two surfaces sharing one conversation store: a chat page and a topology simulator with an embedded tutor panel. The simulator offers ten device types (routers through industrial PLCs), configurable links, an animated virtual terminal (ping, traceroute, ipconfig, ARP, routing), and eleven example topologies; a Node.js backend relays prompts to a cloud provider or a locally hosted open-weight model, so the whole stack runs on commodity hardware without sending student data to third parties.

Two design decisions tie the architecture to pedagogy. A static topology analyzer continuously detects eight classes of misconfiguration (duplicate IPs, isolated nodes, missing gateways, downed links) and renders them to the student, and the identical issue list is serialized into the LLM context, so

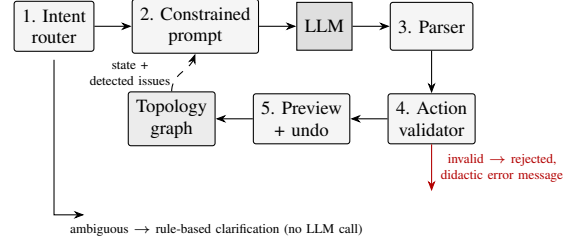


Fig. 1. The validate-before-apply pipeline. The LLM only proposes; deterministic stages decide what reaches the learner’s topology.

TABLE I
SCAFFOLDING CONFIGURATIONS (LEVEL $\times$ FOCUS)

Level	Tutor behavior (applies to both foci)
Guided	Step-by-step; analogies; no unexplained acronyms; flags novice errors; ends with a reflection question.
Balanced	Direct and practical; at most three paragraphs or a short list.
Technical	Dense, standards-referenced (RFC/IEEE); peer register.
Focus	<i>Design</i> : propose and build topologies. <i>Troubleshoot</i> : analyze the graph and explain faults.

tutor and student reason over the same evidence. Conversation state also persists across both surfaces.

B. Adaptive Scaffolding: the Level $\times$ Focus Matrix

Tutoring style is the product of a three-step *Level* and a two-value *Focus* (Table I), yielding six configurations compiled into the system prompt. Levels encode graduated, fadeable scaffolding [6], [7]—from a guided novice register that bans unexplained acronyms and closes each turn with a reflection question, to a technical peer register that cites RFC/IEEE standards—while Focus selects the task frame, design or troubleshoot. Decoding temperature is additionally bound to intent—0.0 for modifications, 0.2 for conceptual explanation—so creative variability is spent on pedagogy, never on state changes.

C. Deterministic Intent Routing and Clarification

Before any model call, a rule-based classifier assigns each utterance one of six intents (conceptual, graph query, diagnostic, clear or ambiguous modification, full-graph application). It uses verb lexicons, question patterns, and the live graph: a message is a *clear* modification only if it references an existing node or concrete target. Ambiguous requests (“add a device somewhere”) never reach the LLM; a clarification dialogue is instead generated from the graph, offering the available attachment points. This keeps such interactions instant, free, and predictable [8], and sets downstream policy: prompt blocks, temperature, and whether the action grammar is enabled.

D. Action Grammar and the Validation Layer

All state changes use a closed grammar of seven actions (add, update, and delete for nodes and links; set link status; apply graph) serialized as JSON inside explicit [CAPA8_ACTION] delimiters. Three prompt-side mechanisms raise the chance an action request yields well-formed

blocks: a directive with paired wrong/correct counter-examples (gateway repair must patch a node property, never add a cable), canonical examples seeded only for action intents, and a single stricter retry when an expected action is missing.

What the model returns is still untrusted. A parser isolates the delimited blocks (malformed JSON is marked invalid and can never execute), and each surviving action passes semantic validation against the current graph: the action type must be whitelisted; referenced nodes must exist; duplicate IPs and links are flagged; unknown device types are normalized through an alias table; and an `add_node` naming an existing node is rewritten into an `update_node`, converting the commonest duplication hallucination into the action the model should have chosen. Rejected actions are not dropped silently: the validator emits an explanatory message (“source node *Router5* not found”), turning the failure into feedback for the student and the model.

E. Human-in-the-Loop Application

Validated actions still require consent at the right granularity. Batches of three or more actions open a preview panel for review and confirmation; any applied batch is one atomic undo step; and the frontend ignores model-proposed coordinates, keeping layout authority on the client. Following human–AI interaction guidance on efficient correction and graceful degradation [10], the tutor’s agency is visible, bounded, and reversible—preconditions, we argue, for trusting an acting tutor in a classroom.

IV. PRELIMINARY EVALUATION

A. Unit-Test Coverage of the Deterministic Pipeline

The deterministic pipeline components are covered by 47 unit tests (validator, intent router, response parser, graph model), all passing. The validator tests exercise the properties the reliability argument rests on: nonexistent references and unknown action/device types are rejected, alias normalization and the `add_node`-to-`update_node` rewrite behave as specified, and duplicate IPs raise warnings. These are unit tests, not formal proofs, but they pin down the deterministic behavior the by-construction interception argument below relies on.

B. Reliability Benchmark

We probed the pipeline with a scripted benchmark of 30 student-style Spanish utterances against a fixed five-node topology seeded with deliberate faults (a PC without gateway, a PC without IP, an in-use IP). The suite has 16 clear modification requests (including gateway-repair, IP-assignment, nonexistent-device, and occupied-IP traps), 6 diagnostic requests, and 8 conceptual or query utterances for which any emitted action is spurious. Every response was parsed and validated by the production code itself, and the full suite was repeated five times per configuration (900 interactions in all); tables report mean $\pm$ SD across the five runs. In every run, all 16 modification prompts yielded a parseable action and every block was well-formed JSON (action-emission and

TABLE II
FULL PIPELINE ACROSS DEPLOYMENT SCENARIOS (30 PROMPTS $\times$ 5 RUNS)

Model	Deploy.	Spur. %	Valid %	Interc.
Gemma 3 4B	Local	37.5 $\pm$ 0.0	77.8 $\pm$ 0.0	10 $\pm$ 0.0
Gemma 3 12B	Local	0.0 $\pm$ 0.0	66.7 $\pm$ 0.0	12 $\pm$ 0.0
Llama 3.3 70B	Cloud	0.0 $\pm$ 0.0	80.0 $\pm$ 0.0	11 $\pm$ 0.0

Mean $\pm$ SD over five runs. Spur.: % of the 8 conceptual/query prompts wrongly answered with an action. Valid: % of parsed actions passing semantic validation. Interc.: invalid actions caught before reaching the graph. Action-emission and JSON-validity rates were 100% in every run (omitted). The three scenarios confound model family, size, and host; we read them as deployment options, not a controlled capability comparison (see §V).

JSON-validity were 100% throughout); we omit these non-discriminating columns and report the three quantities that vary—spurious-action rate, semantic validity, and intercepted actions.

Across deployment scenarios: We ran the full pipeline under three scenarios spanning the privacy/cost spectrum a school faces: a small local model (Gemma 3 4B [16]), a larger local model (Gemma 3 12B), both served through Ollama on commodity hardware, and a cloud-hosted high-capacity model (Llama 3.3 70B via Groq). Because referential validation cannot admit a dangling reference, interception is guaranteed *by construction*; the benchmark therefore does not test *whether* invalid actions are caught but measures the premise that justifies the layer—the rate and classes of actions it must absorb (Table II). None of the three models is free of *semantically* invalid actions: each produced roughly ten to twelve that referenced a nonexistent node or duplicated an address, all intercepted, and even the 70B cloud model did so on every run. What capability changes is the *second* failure mode, the one validation cannot cover: an action that is well-formed and references real nodes yet is contextually *spurious*—emitted in reply to a purely conceptual question. The 4B model does this on 37.5% of conceptual prompts; the 12B and 70B never do. Such actions pass every structural check and would reach the learner’s graph but for the human-confirmable preview and atomic undo of Section III-E; the two safeguards are complementary, and a smaller model shifts load from the automatic one onto the human one (§V).

Ablation of the prompt-side mechanisms: To isolate each prompt-side mechanism we ablated them on the smallest and hardest case, Gemma 3 4B, where structured-output reliability is most fragile: the full pipeline, no seeded examples, no auto-retry, and the directive moved from the system channel into the user prompt (Table III). Routing the directive through the system channel is the dominant aid: removing it is the largest single degradation in the table, maximizing the count of invalid actions the validator must intercept and minimizing semantic validity. The seeded examples and auto-retry show no clear benefit at this model size—the model already emits an action for every request, so the retry never fires—a candid result we report rather than overstate. The validator’s interception nevertheless holds across all four conditions and all runs.

TABLE III
ABLATION OF PROMPT-SIDE MECHANISMS, GEMMA 3 4B LOCAL (30
PROMPTS $\times$ 5 RUNS PER CONDITION)

Condition	Spur. %	Valid %	Interc.
Full pipeline	37.5 $\pm$ 0.0	77.8 $\pm$ 0.0	10 $\pm$ 0.0
– seeded examples	37.5 $\pm$ 0.0	92.0 $\pm$ 0.0	4 $\pm$ 0.0
– auto-retry	37.5 $\pm$ 0.0	76.6 $\pm$ 0.0	11 $\pm$ 0.0
– system channel	37.5 $\pm$ 0.0	68.9 $\pm$ 0.0	14 $\pm$ 0.0

Mean $\pm$ SD over five runs. Columns as in Table II. Action-emission and JSON-validity rates were 100% in every condition and run (omitted). Interc.: invalid actions intercepted by the validator that would otherwise have corrupted the graph.

V. DISCUSSION AND LIMITATIONS

The validation layer guarantees *structural* soundness, not pedagogical optimality, and two failure modes survive it. First, an action can be structurally valid yet didactically wrong: connecting PC1 to the router with a new cable (the gateway trap) is referentially valid but pedagogically incorrect, and only the counter-example prompts, not the validator, steer the model away from it; encoding such domain-pedagogical rules as first-class validations is a natural next step. Second, the spurious-action mode quantified above (37.5% of conceptual prompts for the 4B model) is structurally valid by definition and passes the validator untouched, so only the human-in-the-loop preview and undo stand in its way. A validator that silently absorbs every malformed action also risks *automation complacency*—a student who never sees a rejection may trust whatever is applied—which is why CAPA8 surfaces rejections as visible, didactic events rather than silent drops.

The benchmark has clear scope limits: it covers three models, one seeded topology, and Spanish utterances only, and it measures reliability, not learning. The three deployment scenarios moreover confound model family, size, and host (Gemma vs. Llama; 4B/12B vs. 70B; local vs. cloud), so they should be read as deployment options a school might select rather than a controlled capability comparison; isolating any single factor would need a crossed design we did not run. No claims about learning outcomes are made; comprehension, trust calibration, and scaffolding-level effects require the classroom study below. Finally, we *conjecture*—without yet testing it—that validate-before-apply extends to other artifact-centric subjects (electronic circuits, database schemas, chemical process diagrams), wherever an LLM tutor may touch the thing the student is building.

VI. CONCLUSION AND FUTURE WORK

CAPA8 demonstrates that an LLM tutor can be given hands—the ability to construct and repair the learner’s network topology—without inheriting the model’s unreliability, by interposing deterministic intent routing, a closed action grammar, state-aware semantic validation, and human-confirmable application between proposal and effect. The interception of structurally invalid actions is guaranteed by construction; our preliminary technical evidence (47 passing unit tests; a 900-interaction benchmark spanning local 4B and 12B models and a 70B cloud model, five runs each) instead characterizes what that layer must absorb: every model, the 70B cloud

model included, proposes invalid actions on every run, while smaller models additionally emit spurious actions on conceptual prompts—a class structural validation cannot see and that the human-in-the-loop step exists to cover. The architecture runs on hardware schools already own. The next step is a controlled classroom study with networking students (pre/post concept inventory, usability, and scaffolding-level comparison), to be conducted under institutional ethics review. This paper reports no human-subjects data.

ACKNOWLEDGMENT

AI-use disclosure (per IEEE/TALE policy): a generative AI system (Claude, Anthropic) assisted with text editing, benchmark scripting, and LaTeX formatting under the authors’ direction; the authors reviewed and verified all content, code, and results, take full responsibility, and designed and implemented CAPA8 themselves. [Author names, affiliations, funding, and the code repository are withheld for double-blind review and restored upon acceptance.]

REFERENCES

- [1] E. Kasneci *et al.*, “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [2] P. Denny *et al.*, “Computing education in the era of generative AI,” *Communications of the ACM*, vol. 67, no. 2, pp. 56–67, 2024.
- [3] M. Liffiton, B. Sheese, J. Savelka, and P. Denny, “CodeHelp: Using large language models with guardrails for scalable support in programming classes,” in *Proc. 23rd Koli Calling Int. Conf. on Computing Education Research*, 2023, pp. 1–11.
- [4] Z. Ji *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [5] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press, 1978.
- [6] D. Wood, J. S. Bruner, and G. Ross, “The role of tutoring in problem solving,” *Journal of Child Psychology and Psychiatry*, vol. 17, no. 2, pp. 89–100, 1976.
- [7] B. J. Reiser, “Scaffolding complex learning: The mechanisms of structuring and problematizing student work,” *Journal of the Learning Sciences*, vol. 13, no. 3, pp. 273–304, 2004.
- [8] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [9] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” in *Proc. EMNLP 2023: System Demonstrations*, 2023, pp. 431–445.
- [10] S. Amershi *et al.*, “Guidelines for human–AI interaction,” in *Proc. CHI Conf. on Human Factors in Computing Systems*, 2019, pp. 1–13.
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.
- [12] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” *arXiv preprint arXiv:2307.16789*, 2023.
- [13] H. Jin, S. Lee, H. Shin, and J. Kim, “Teach AI how to code: Using large language models as teachable agents for programming education,” in *Proc. CHI Conf. on Human Factors in Computing Systems*, 2024, pp. 1–28.
- [14] Cisco Networking Academy, “Cisco Packet Tracer,” 2024. [Online]. Available: <https://www.netacad.com/courses/packet-tracer>
- [15] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010, pp. 1–6.
- [16] Gemma Team, Google DeepMind, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025.